Cow Protocol: Euler Integration

Security Review

Cantina Managed review by:
Slowfi, Lead Security Researcher
Cryptara, Security Researcher

March 7, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 High Risk . . . 4
3.1.1 EIP-712 Type String Mismatch for `CollateralSwapParams` . . . 4
3.2 Medium Risk . . . 4
3.2.1 Silent Successful Execution When Next Wrapper Is an EOA . . . 4
3.2.2 `ecrecover` Return Value Not Checked . . . 5
3.2.3 Stale Nonce Embedded in EVC Permit Calldata Breaks Wrappers That Require Permission in Both Phases . . . 6
3.3 Low Risk . . . 7
3.3.1 `verifyAndBuildWrapperData` Silently Omits Two Promised Safety Checks . . . 7
3.3.2 Pre-Approved Hash Flow Silently Revokes Operator Authorization Without Documentation . . . 7
3.3.3 `_evcInternalSettle` reverts when `asset` equals `collateralVault` . . . 8
3.3.4 Missing onchain signal when close results in a partial repay . . . 9
3.4 Gas Optimization . . . 9
3.4.1 Unused constants . . . 9
3.5 Informational . . . 9
3.5.1 `CowEvcCollateralSwapWrapper` Never Disables the Old Collateral Vault . . . 9
3.5.2 Unrestored Memory Slot After In-Place EIP-712 Hash in `Inbox.isValidSignature` 10
3.5.3 Events Emit User-Signed Minimum Amounts Instead of Actual Executed Amounts . . 11
3.5.4 `InboxFactory` Does Not Validate EVC Subaccount Relationship . . . 11
3.5.5 `setPreApprovedHash(false)` Transitions Unapproved Hashes Directly to `CONSUMED` 12
3.5.6 Error Is Defined but Never Thrown . . . 13
3.5.7 Interest Accrual Between Signing and Execution Can Leave a Residual Debt After Close 13
3.5.8 `wrappedSettle` does not bind signed wrapper parameters to `settleData` . . . 14
3.5.9 `_evcInternalSettle` emits `collateralAmount` as input not collateral spent . . . 15
3.5.10 `_evcInternalSettle` swap output check can pass without any settlement output . 15
3.5.11 `Inbox` hardcodes settlement domain separator . . . 15
3.5.12 Missing Euler vault validation in open/close wrappers . . . 16
3.5.13 Unconditional max approval to the vault relayer . . . 16
3.5.14 `toAmount` parameter in `CowEvcCollateralSwapWrapper` is unused . . . 17
3.5.15 No interfaces provided for integrators . . . 17

DRAFT

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

DRAFT

2 Security Review Summary

From Feb 22nd to Feb 27th the Cantina team conducted a review of euler-integration-contracts on commit hash dcb908c9. The team identified a total of **24** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	3	3	0
Low Risk	4	4	0
Gas Optimizations	1	1	0
Informational	15	12	3
Total	**24**	**21**	**3**

The Cantina Managed team reviewed Cow Protocol's euler-integration-contracts holistically on commit hash beb87e83 and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for euler-integration-contracts on commit hash dcb908c9:

```
src
├── CowEvcBaseWrapper.sol
├── CowEvcClosePositionWrapper.sol
├── CowEvcCollateralSwapWrapper.sol
├── CowEvcOpenPositionWrapper.sol
├── CowWrapper.sol
├── CowWrapperHelpers.sol
├── Inbox.sol
├── InboxFactory.sol
└── PreApprovedHashes.sol
```

DRAFT

3 Findings

3.1 High Risk

3.1.1 EIP-712 Type String Mismatch for `CollateralSwapParams`

Severity: High Risk

Context: *(No context files were provided by the reviewer)*

Description: In `CowEvcCollateralSwapWrapper.sol`, the `PARAMS_TYPE_HASH` is computed from a type string that declares a single field `uint256 swapAmount`:

```
PARAMS_TYPE_HASH = keccak256(
    "CollateralSwapParams(address owner,address account,uint256 deadline,address
    ↪  fromVault,address toVault,uint256 swapAmount)"
);
```

However, the actual `CollateralSwapParams` struct contains **two** separate amount fields:

```
struct CollateralSwapParams {
    address owner;
    address account;
    uint256 deadline;
    address fromVault;
    address toVault;
    uint256 fromAmount;  // ← not in type string
    uint256 toAmount;    // ← not in type string
}
```

Under EIP-712, the `structHash` is computed as `keccak256(typeHash || encodeData(struct))`. The `encodeData` output is derived from the actual struct memory layout (7 fields, 7 × 32 = 224 bytes), but the `typeHash` describes only 6 fields. This mismatch means:

1. The `PARAMS_TYPE_HASH` itself is computed from the wrong string, producing a different hash than what the struct encoding would require.
2. Off-chain tooling following the EIP-712 spec will compute the struct hash using `swapAmount` from the type string, which does not map to any single field. The hashes will never match the on-chain computation.
3. Any `fromAmount/toAmount` values committed by the user are not properly bound by the EIP-712 signature, undermining the integrity guarantee of the pre-approved hash authorization flow.

Recommendation: Align the type string exactly with the struct definition:

```
PARAMS_TYPE_HASH = keccak256(
    "CollateralSwapParams(address owner,address account,uint256 deadline,address
    ↪  fromVault,address toVault,uint256 fromAmount,uint256 toAmount)"
);
```

Consider adding a compile-time or constructor-time assertion that `PARAMS_SIZE` (derived via `abi.encode(...).length`) equals `(number of fields in type string) * 32` to catch future drift between the struct and its type string.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Silent Successful Execution When Next Wrapper Is an EOA

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

DRAFT

Description: In `CowWrapper.sol`, the `_next` function dispatches to the next wrapper in the chain using a high-level call:

```
CowWrapper(nextWrapper).wrappedSettle(settleData, remainingWrapperData);
```

No check is performed to verify that `nextWrapper` has deployed code, implements the `ICowWrapper` interface, or is associated with the expected settlement contract. In the EVM, a `call` to an EOA always succeeds and returns empty data. Because `wrappedSettle` is declared `void` (returns no value), the Solidity-generated call does not verify any return data. The call therefore completes successfully while executing zero wrapper logic - the custom pre/post-settlement operations of the intended wrapper are silently skipped, but the settlement continues.

Recommendation: Require a recognizable return value from each `wrappedSettle` call to confirm the callee executed real wrapper logic. For example:

```
// Define a magic value
bytes4 constant WRAPPED_SETTLE_MAGIC = bytes4(keccak256("wrappedSettle()"));

// In ICowWrapper / CowWrapper
function wrappedSettle(...) external returns (bytes4);

// In _next
bytes4 result = CowWrapper(nextWrapper).wrappedSettle(settleData, remainingWrapperData);
require(result == WRAPPED_SETTLE_MAGIC, InvalidWrapper(nextWrapper));
```

This ensures that a call to an EOA (which returns empty bytes, not the magic value) is detected and reverted, preventing silent bypass of wrapper logic.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.2.2 `ecrecover` Return Value Not Checked

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: In `Inbox.sol`, after recovering the signer from an ECDSA signature, the result is compared directly against `BENEFICIARY` with no guard against the zero address:

```
address signer = ecrecover(inboxOrderDigest, v, r, s);
require(signer == BENEFICIARY, Unauthorized(signer));
```

`ecrecover` is a precompile that returns `address(0)` for any malformed or invalid signature (e.g., `s` out of range, invalid `v`, all-zero inputs). The check `signer == BENEFICIARY` only protects correctly if `BENEFICIARY != address(0)`. The `Inbox` constructor accepts `beneficiary` without a zero-address guard:

```
constructor(address executor, address beneficiary, address settlement) {
    OPERATOR = executor;
    BENEFICIARY = beneficiary; // no require(beneficiary != address(0))
//...
}
```

If an `Inbox` is deployed - intentionally or by mistake - with `beneficiary = address(0)`, then any caller can pass an entirely invalid signature (e.g., 65 zero bytes) and `isValidSignature` will return the EIP-1271 magic value, granting unrestricted order signing authority on behalf of that `Inbox`. Since `Inbox` holds user funds and acts as the CoW order signer for the close-position flow, this would allow an attacker to authorize arbitrary orders draining all tokens held in the contract.

Recommendation: Replace the raw `ecrecover` call with OpenZeppelin's `ECDSA.recover`, which already reverts on an invalid signature (including the `address(0)` case) and handles signature malleability:

```
import {ECDSA} from "openzeppelin/contracts/utils/cryptography/ECDSA.sol";
```

DRAFT

```
// reverts with ECDSA.ECDSAInvalidSignature if signer == address(0)
address signer = ECDSA.recover(inboxOrderDigest, v, r, s);
require(signer == BENEFICIARY, Unauthorized(signer));
```

The library is already available as a transitive dependency via the EVC. This single change eliminates both the zero-address footgun and any future signature malleability concerns without requiring a separate constructor guard.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.2.3 Stale Nonce Embedded in EVC Permit Calldata Breaks Wrappers That Require Permission in Both Phases

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The wrapper constructs EVC batch calldata by wrapping certain `addItems` segments inside an `EVC.permit` call. Inside `_addEvcBatchItems`, the current nonce is read at batch-construction time via `EVC.getNonce(...)` and encoded directly into the permit calldata:

```
data: abi.encodeCall(
    IEVC.permit,
    (
        owner,
        address(this),
        uint256(NONCE_NAMESPACE),
        EVC.getNonce(bytes19(bytes20(owner)), NONCE_NAMESPACE), // ← read here
        deadline,
        0,
        _encodePermitData(addItems, param),
        signature
    )
)
```

`_invokeEvc` calls `_addEvcBatchItems` twice - once for `_encodeBatchItemsBefore` and once for `_encodeBatchItemsAfter` - both before `EVC.batch(items)` is invoked:

```
itemIndex = _addEvcBatchItems(items, partialItems, itemIndex, owner, deadline,
    needsPermission ? signature : new bytes(0), param); // nonce N embedded

// ... settlement callback item added ...

itemIndex = _addEvcBatchItems(items, partialItems, itemIndex, owner, deadline,
    needsPermission ? signature : new bytes(0), param); // nonce N embedded again
```

Because both `getNonce` reads happen before any batch execution, both permit items embed the same nonce value `N`. When `EVC.batch` later executes:

1. First permit item is processed → nonce verified as `N` → nonce incremented to `N+1` → success.
2. Second permit item is processed → nonce still encoded as `N` → current nonce is `N+1` → revert.

Any wrapper that overrides both `_encodeBatchItemsBefore` and `_encodeBatchItemsAfter` returning `needsPermission = true` will therefore always revert in the permit flow. Additionally, since `_invokeEvc` accepts only a single `signature`, there is no way to supply two independent permit signatures for the two phases anyway.

The current production wrappers (`CowEvcOpenPositionWrapper`, `CowEvcClosePositionWrapper`, `CowEvcCollateralSwapWrapper`) avoid this by design - only one of the two phases requests permission. However, the abstract base contract does not enforce this constraint, and any future wrapper that legitimately needs both pre- and post-settlement EVC operations authorized via permit will hit this bug immediately.

DRAFT

Recommendation: Enforce at the base contract level that at most one phase may require a permit signature, by adding a validation in `_invokeEvc`:

```
(partialItemsBefore, needsBefore) = _encodeBatchItemsBefore(param);
(partialItemsAfter,  needsAfter)  = _encodeBatchItemsAfter(param);
require(!(needsBefore && needsAfter), BothPhasesRequirePermission());
```

Alternatively, if dual-phase authorization is ever needed, each phase must use an independent nonce namespace or the nonce must be read lazily inside the EVC batch execution context rather than encoded at construction time.

Cow Protocol: Fixed in commit f19449a3.

Cantina Managed: Fix verified.

3.3 Low Risk

3.3.1 `verifyAndBuildWrapperData` Silently Omits Two Promised Safety Checks

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The NatSpec for `verifyAndBuildWrapperData` in `CowWrapperHelpers.sol` documents four validations:

```
/// 1. Verifies each wrapper is authenticated via WRAPPER_AUTHENTICATOR   □ implemented
/// 2. Verifies each wrapper's data is valid via validateWrapperData       □ implemented
/// 3. Verifies all wrappers use the same settlement contract              □ NOT
↪  implemented
/// 4. Verifies the settlement contract is not authenticated as a solver   □ NOT
↪  implemented
```

The custom errors for the missing checks are defined in the contract but thrown nowhere in the codebase:

```
error SettlementContractShouldNotBeSolver(address settlementContract, address
↪  authenticatorContract);
error SettlementMismatch(uint256 wrapperIndex, address expectedSettlement, address
↪  actualSettlement);
```

Beyond the missing checks, the function also does not verify that each `wrapperCalls[i].target` is a deployed contract (`code.length > 0`). Because the `WRAPPER_AUTHENTICATOR` may mark any address as a solver - including EOAs - an EOA can pass the authentication check and be accepted as a valid wrapper target by this helper. Downstream consumers relying on this helper to validate a chain before execution may be misled into believing a fully-validated chain has been produced.

Recommendation: Implement the missing validations inside `verifyAndBuildWrapperData`:

1. Settlement consistency: Read `ICowWrapper(target).SETTLEMENT()` for each wrapper and assert it matches the first wrapper's settlement contract, reverting with `SettlementMismatch`.
2. Settlement not a solver: After collecting the settlement address, assert `!WRAPPER_AUTHENTICATOR.isSolver(settlement)`, reverting with `SettlementContractShouldNotBeSolver`.
3. Code existence: Assert `wrapperCalls[i].target.code.length > 0` before calling `validateWrapperData`, to ensure the target is a deployed contract and not an EOA silently accepted by the authenticator.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.3.2 Pre-Approved Hash Flow Silently Revokes Operator Authorization Without Documentation

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

DRAFT

Description: In `CowEvcBaseWrapper.sol`, after executing an operation via the pre-approved hash flow (i.e., `signature.length == 0`), the contract automatically revokes its own operator access on both the `owner` and `account` EVC sub-accounts:

```
if (signature.length == 0) {
    uint256 mask = EVC.getOperator(bytes19(bytes20(owner)), address(this));

    if ((mask & (1 << (uint160(owner) ^ uint160(account)))) > 0) {
        EVC.setAccountOperator(account, address(this), false);
    }
    if (owner != account && mask & 1 > 0) {
        EVC.setAccountOperator(owner, address(this), false);
    }
}
```

This behavior is not documented in the NatSpec, the README, or any interface-level comments. Any integration or user that grants persistent operator authorization to the wrapper (e.g., to allow multiple sequential operations) will find their authorization silently revoked after the first successful pre-approved hash execution. The next invocation will fail mid-execution inside the EVC batch when the wrapper attempts to operate on behalf of the user, resulting in a confusing and hard-to-diagnose revert.

Recommendation: Add explicit NatSpec documentation on `_invokeEvc` and the pre-approved hash flow clearly stating that operator authorization is revoked as part of the flow:

```
/// @dev When using the pre-approved hash flow (empty signature), this function will
/// revoke operator access for both `owner` and `account` after execution completes.
/// Integrations relying on persistent operator authorization MUST re-grant access
/// before the next operation.
```

Additionally, consider making this behavior configurable - for example, by including a flag in the `params` struct (e.g., `bool revokeOperatorAfterExecution`) so that users or integrations that need persistent operator access can opt out of the automatic revocation.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.3.3 `_evcInternalSettle` reverts when `asset` equals `collateralVault`

Severity: Low Risk

Context: CowEvcClosePositionWrapper.sol#L195-L208

Description: The function `_evcInternalSettle` from contract `CowEvcClosePositionWrapper` computes `swapSourceBalance` as the Inbox balance of `params.collateralVault` and `swapResultBalance` as the Inbox balance of `IERC4626(params.borrowVault).asset()` after executing the CoW settlement.

It then transfers `swapSourceBalance` (the remaining source collateral) back to `params.account` before performing the debt repayment, while still using the pre-transfer `swapResultBalance` to derive `repayAmount`.

When `IERC4626(params.borrowVault).asset()` is equal to `params.collateralVault`, `swapSourceBalance` and `swapResultBalance` refer to the same token balance. As a result, the transfer of `swapSourceBalance` drains the token balance that is later assumed to be available for the excess buy token transfer and/or `IBorrowing(params.borrowVault).repay`, causing the close flow to revert.

Closing (or reducing) a position through `CowEvcClosePositionWrapper` reverts for configurations where the borrowed asset equals the collateral vault token, preventing this wrapper from being used for those positions.

Recommendation:

Consider to validate that `IERC4626(params.borrowVault).asset()` is different from `params.collateralVault`, or consider to handle the equality case explicitly, for example, repay before returning any source balance and compute leftovers after repayment.

DRAFT

Cow Protocol: Fixed in commit beb87e83.

Cantina Managed: Fix verified.

3.3.4 Missing onchain signal when close results in a partial repay

Severity: Low Risk

Context: CowEvcClosePositionWrapper.sol#L196

Description: The close flow in `CowEvcClosePositionWrapper` is intentionally designed **not** to revert if the position is not fully closed (to avoid turning debt drift / state changes into a potential batch-wide DoS vector through CoW Settlement batching).

However, when the execution ends up repaying less than the live debt (e.g., due to accrued interest or other debt changes between order construction and execution), the transaction can still succeed while leaving a residual debt open. In that case, the contract does not emit any on-chain signal indicating that the "close" was partial.

This makes it difficult for:

- Frontends to reliably inform users that their position remains open, and...
- Monitoring/automation to detect that follow-up action is required.

Recommendation: Consider to emit an event that indicates whether the close fully repaid the debt, for example including:

- `owner`, `account`, `borrowVault`, `collateralVault`.
- `debtAmount` (measured at the time repayment is attempted).
- `repaidAmount`.
- `residualDebt` (or a boolean `fullyClosed`).

This provides a low overhead integration hook for UIs and indexers while preserving the non-reverting design choice. Also could consider reverting if position is not closed.

Cow Protocol: Fixed in commit beb87e83.

Cantina Managed: Fix verified.

3.4 Gas Optimization

3.4.1 Unused constants

Severity: Gas Optimization

Context: CowEvcBaseWrapper.sol#L22-L29

Description: The contract `CowEvcBaseWrapper` defines the constants `KIND_SELL` and `KIND_BUY`.

These constants are not referenced anywhere in the repository code paths that inherit from `CowEvcBaseWrapper`, and `CowEvcBaseWrapper` itself does not compute or validate CoW order struct hashes. As a result, the constants and their. associated comments are dead code.

Increased maintenance surface and reduced clarity around what data is actually validated onchain.

Recommendation: Consider to remove `KIND_SELL` and `KIND_BUY` and the related comments, or consider to use them in the code path that computes or validates an order struct hash if that is intended.

Cow Protocol: Fixed in commit d15f7541.

Cantina Managed: Fix verified.

3.5 Informational

3.5.1 `CowEvcCollateralSwapWrapper` Never Disables the Old Collateral Vault

Severity: Informational

DRAFT

Context: *(No context files were provided by the reviewer)*

Description: In `CowEvcCollateralSwapWrapper.sol`, the wrapper enables the new collateral vault (`toVault`) for the user's account. However, `disableCollateral` for the old collateral vault (`fromVault`) is never called - the function is absent from the entire codebase.

After a collateral swap, the old vault remains enabled as collateral indefinitely. A user who has fully exited `fromVault` still has it registered as collateral in the EVC, increasing their theoretical collateral exposure without benefit. More significantly, if someone later deposits funds into `fromVault` for this account for an unrelated reason, those funds become encumbered as collateral unexpectedly. The same limitation affects `CowEvcClosePositionWrapper`, which cannot call `disableController` because the EVC permit flow does not support signing post-settlement operations.

Recommendation: Document this behavior explicitly in the NatSpec of `CowEvcCollateralSwapWrapper` to ensure integrators and users understand that disabling the old collateral vault is a required post-swap step. Consider emitting a dedicated event (e.g., `CollateralDisableRequired`) or providing a helper function that checks whether `fromVault` balance is zero and, if so, calls `EVC.disableCollateral` automatically. This would reduce the risk of silently leaving stale collateral registrations.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.5.2 Unrestored Memory Slot After In-Place EIP-712 Hash in `Inbox.isValidSignature`

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: In `Inbox.sol`, the `isValidSignature` function allocates a `bytes memory orderData` array from calldata and then overwrites its length word in assembly to perform an in-place EIP-712 struct hash:

```
assembly {
    mstore(orderData, typeHash)            // overwrites the length slot
    structHash := keccak256(orderData, 416)
}
```

Unlike the analogous pattern in `CowEvcBaseWrapper.sol`, which saves and restores the overwritten word:

```
let wordBeforeParam := mload(wordBeforeParamPtr)
mstore(wordBeforeParamPtr, typeHash)
structHash := keccak256(...)
mstore(wordBeforeParamPtr, wordBeforeParam)   // ← restored
```

`Inbox.sol` never restores the original length value of `orderData`. Currently this is not exploitable because `orderData` is not referenced after the assembly block. However, it constitutes a latent memory corruption hazard: any future refactor that reads or passes `orderData` after this block would silently operate on a bytes array whose stored length has been permanently replaced with `typeHash` (a 32-byte hash value). This could cause incorrect length-based calculations, unexpected reverts, or subtle security bugs.

Recommendation: Restore the original length slot after computing the struct hash, following the same pattern already used in `CowEvcBaseWrapper.sol`:

```
assembly {
    let originalLength := mload(orderData)
    mstore(orderData, typeHash)
    structHash := keccak256(orderData, 416)
    mstore(orderData, originalLength)   // restore
}
```

Alternatively, add a prominent inline comment making clear that `orderData` must never be used after this assembly block, to prevent future regressions.

Cow Protocol: Fixed in PR 24.

DRAFT

Cantina Managed: Fix verified.

3.5.3 Events Emit User-Signed Minimum Amounts Instead of Actual Executed Amounts

Severity: Informational

Context: *(No context files were provided by the reviewer)*

In `CowEvcOpenPositionWrapper.sol` and `CowEvcCollateralSwapWrapper.sol`, the `CowEvcPositionOpened` and `CowEvcCollateralSwapped` events are emitted with parameter values drawn directly from the user-signed `params` struct:

```
emit CowEvcPositionOpened(
    params.owner,
    params.account,
    params.collateralVault,
    params.borrowVault,
    params.collateralAmount,   // ← minimum signed amount, not actual
    params.borrowAmount        // ← minimum signed amount, not actual
);
```

In a CoW Protocol sell order, `collateralAmount` and `borrowAmount` represent the minimum acceptable amounts specified by the user, not the amounts actually received after settlement. The solver may provide a better rate, resulting in larger actual amounts. Off-chain indexers, analytics dashboards, and monitoring systems that interpret these events as recording actual position sizes will systematically underreport leverage and collateral values.

Recommendation: Document clearly in the event NatSpec that `collateralAmount` and `borrowAmount` in the emitted events reflect user-signed minimum values and not actual execution amounts:

```
/// @notice Emitted when a position is opened.
/// @param collateralAmount The minimum collateral amount specified by the user (NOT the
↪  actual deposited amount)
/// @param borrowAmount The minimum borrow amount specified by the user (NOT the actual
↪  borrowed amount)
event CowEvcPositionOpened(...)
```

If on-chain tracking of actual amounts is required in the future, a post-settlement balance snapshot approach (recording vault shares before and after the `_next` call) could be introduced.

Cow Protocol: Fixed in PR 24.

Cantina Managed: Fix verified.

3.5.4 `InboxFactory` Does Not Validate EVC Subaccount Relationship

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `InboxFactory.getInbox` and its internal counterpart `_getInbox` accept arbitrary `owner` and `subaccount` addresses and deploy an `Inbox` contract for them without verifying that `subaccount` is a valid EVC subaccount of `owner`. In the EVC, a valid subaccount must share the same highest 19 bytes (bytes19 prefix) as its owner. `CowEvcBaseWrapper._invokeEvc` enforces this invariant at execution time:

```
require(bytes19(bytes20(owner)) == bytes19(bytes20(account)),
↪  SubaccountMustBeControlledByOwner(account, owner));
```

However, no equivalent guard exists in the factory:

```
// InboxFactory._getInboxAddress — no bytes19 check
function _getInboxAddress(address owner, address subaccount) internal view
    returns (address creationAddress, bytes memory creationCode, bytes32 salt)
{
    salt = bytes32(uint256(uint160(subaccount)));
```

DRAFT

```
    creationCode = abi.encodePacked(type(Inbox).creationCode, abi.encode(address(this),
    ↪  owner, INBOX_SETTLEMENT));
    creationAddress = Create2.computeAddress(salt, keccak256(creationCode));
}
```

Because `getInbox` is `external`, anyone can trigger an `Inbox` deployment for an invalid `(owner, subaccount)` pair where the two addresses share no EVC subaccount relationship. The resulting `Inbox` has `BENEFICIARY = owner` and `OPERATOR = address(this)` (the wrapper), but any wrapper invocation using that pair will revert in `_invokeEvc` before the inbox is actually used. The inbox is therefore deployed and permanently unoperatable through the wrapper, though the `BENEFICIARY` retains direct recovery access via `callTransfer`.

Recommendation: Add the same bytes19 prefix check already present in `_invokeEvc` to both the public `getInbox` entry point and the internal `_getInbox`, so that inbox deployment is only permitted for valid EVC subaccount pairs:

```
require(
    bytes19(bytes20(owner)) == bytes19(bytes20(subaccount)),
    SubaccountMustBeControlledByOwner(subaccount, owner)
);
```

Cow Protocol: Fixed in commit 60160d9.

Cantina Managed: Fix verified.

3.5.5 `setPreApprovedHash(false)` Transitions Unapproved Hashes Directly to `CONSUMED`

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `PreApprovedHashes.sol` implements a three-state machine for operation hashes:

```
0 (unset) → PRE_APPROVED → CONSUMED
```

The `setPreApprovedHash` function's revocation branch (`approved = false`) unconditionally writes `CONSUMED` to storage without first verifying that the hash is currently in the `PRE_APPROVED` state:

```
function setPreApprovedHash(bytes32 hash, bool approved) external {
    require(preApprovedHashes[msg.sender][hash] != CONSUMED, AlreadyConsumed(msg.sender,
    ↪  hash));

    if (approved) {
        preApprovedHashes[msg.sender][hash] = PRE_APPROVED;
    } else {
        preApprovedHashes[msg.sender][hash] = CONSUMED; // ← no check on current state
    }
    emit PreApprovedHash(msg.sender, hash, approved);
}
```

A user who calls `setPreApprovedHash(hash, false)` while the hash is still in state `0` (never approved) will silently trigger the direct transition `0 → CONSUMED`. Because `CONSUMED` is a terminal state - the top-level `require` gates any future call with `!= CONSUMED` - the hash is permanently blacklisted from ever being approved. The user loses the ability to use that hash for the pre-approved flow, with no warning and no way to recover.

The intended use of `false` is to revoke a previously granted approval, so the only valid prior state should be `PRE_APPROVED`. Calling revoke on an unapproved hash is almost certainly a user error and should revert with a clear message rather than silently poison the hash slot.

Recommendation: Add an explicit state guard to the `false` branch so that revocation only succeeds from the `PRE_APPROVED` state:

```
} else {
    require(preApprovedHashes[msg.sender][hash] == PRE_APPROVED,
    ↪  HashNotApproved(msg.sender, hash));
```

DRAFT

```
    preApprovedHashes[msg.sender][hash] = CONSUMED;
}
```

This enforces the intended `PRE_APPROVED → CONSUMED` transition and prevents the unintended `0 → CONSUMED` shortcut.

Cow Protocol: Fixed in commit ca32750.

Cantina Managed: Fix verified.

3.5.6 Error Is Defined but Never Thrown

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `CowEvcClosePositionWrapper.sol` declares three custom errors intended for runtime validation of the close-position flow:

```
error NoSwapOutput(address inboxForSwap);                                          // 
↪  thrown
error InsufficientDebt(uint256 expectedMinDebt, uint256 actualDebt);               // 
↪  never thrown
error UnexpectedRepayResult(uint256 expectedRepayAmount, uint256 actualRepaidAmount); //
↪  thrown
```

`NoSwapOutput` and `UnexpectedRepayResult` are both actively used as runtime guards in `_evcInternalSettle`. `InsufficientDebt` is defined with parameters that clearly describe a minimum-debt guard (`expectedMinDebt`, `actualDebt`), but it is thrown nowhere in the codebase.

The intended check would compare the live `debtOf(params.account)` value against a user-specified minimum at execution time:

```
uint256 debtAmount = IBorrowing(params.borrowVault).debtOf(params.account);
// intended: require(debtAmount >= params.minDebt, InsufficientDebt(params.minDebt,
↪  debtAmount));
```

However, `ClosePositionParams` has no `minDebt` field, so the guard could not be implemented even if the revert were added. The struct and the error are misaligned - the error implies a planned feature that was never completed. Without this check, a user cannot set a lower bound on the outstanding debt at the moment of execution. If the account's debt changes significantly between signing and settlement (e.g., due to interest accrual or a concurrent partial repayment), the close will proceed unconditionally against whatever debt remains.

Recommendation: Either remove `InsufficientDebt` if the check is not intended to be implemented, or complete the implementation by adding a `minDebt` field to `ClosePositionParams` and throwing the error when `debtAmount < params.minDebt`. Leaving a defined-but-unused error alongside two actively used sibling errors is misleading and signals incomplete logic to future maintainers.

Cow Protocol: Fixed in commit 215c283.

Cantina Managed: Fix verified.

3.5.7 Interest Accrual Between Signing and Execution Can Leave a Residual Debt After Close

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: In `CowEvcClosePositionWrapper.sol`, the debt at execution time is read live from the vault:

```
uint256 debtAmount = IBorrowing(params.borrowVault).debtOf(params.account);
```

This value is captured **after** the CoW settlement executes, using the debt that has accrued at that point. The wrapper correctly handles the surplus case - when the swap produces more than the current debt, the excess is refunded to the owner:

DRAFT

```
if (repayAmount > debtAmount) {
    inbox.callTransfer(address(borrowAsset), params.owner, repayAmount - debtAmount);
    repayAmount = debtAmount;
}
```

However, there is no symmetric handling for the deficit case. When a user constructs a `KIND_BUY` CoW order with `buyAmount` equal to their debt at signing time, and interest accrues between signing and settlement execution, the swap produces exactly the signed `buyAmount` - which is now less than the grown `debtAmount`. The wrapper silently repays only the partial amount, and the transaction succeeds with a small residual debt remaining open:

```
Signing time:   debt = 5.00 ETH  → user sets buyAmount = 5.00 ETH
Execution time: debt = 5.05 ETH  → swap produces 5.00 ETH
                repayAmount (5.00) < debtAmount (5.05)
                → repays 5.00 ETH, 0.05 ETH residual debt remains
```

The user intended a full position close but ends up with an open debt position they may not be monitoring. Depending on the vault's liquidation parameters, even a small residual debt could eventually pose a liquidation risk if collateral is not adjusted.

Recommendation: Document this behavior prominently in the NatSpec of `ClosePositionParams` and in user-facing integration guides. Off-chain tooling and frontends constructing close orders should account for interest accrual by either:

1. Adding a small buffer to `buyAmount` for `KIND_BUY` orders (e.g., `currentDebt * 1.01`) to absorb interest that accrues before settlement.
2. Using a `KIND_SELL` order instead, which sells a fixed amount of collateral and repays whatever debt asset is received, naturally accommodating interest growth without producing residual debt.
3. Querying the live debt amount as late as possible before order submission to minimise the accrual window.

Cow Protocol: Acknowledged.

Cantina Managed: The documentation was updated in commit 2599574 to reflect the intentions.

3.5.8 `wrappedSettle` does not bind signed wrapper parameters to `settleData`

Severity: Informational

Context: CowWrapper.sol#L146-L160

Description: The function `wrappedSettle` from contract `CowWrapper` authenticates `msg.sender` as a solver and forwards `settleData` to `_wrap` after slicing `chainedWrapperData` by length.

Across wrappers in this repository, user authorization is validated over wrapper parameters through `CowEvcBaseWrapper._invokeEvc` using either a permit signature or a pre-approved hash. However, the wrappers do not decode or validate `settleData` to enforce any relationship between the signed wrapper parameters and the CoW settlement payload. The function `_next` only checks that `settleData` starts with the `ICowSettlement.settle` selector before calling the settlement contract.

Correctness relies on the integration ensuring that the CoW settlement payload is consistent with the user-authorized wrapper parameters. This relationship is not enforced onchain by the wrappers.

Recommendation: Consider to document explicitly that wrapper parameter authorization does not cover `settleData`, and that offchain components must ensure `settleData` matches the intended wrapper parameters. Also could consider to add an optional onchain binding between wrapper parameters and the settlement payload where compatible with the wrapper chaining design.

Cow Protocol: Fixed in commit d2e453f0.

Cantina Managed: Fix verified.

DRAFT

3.5.9 `_evcInternalSettle` emits `collateralAmount` as input not collateral spent

Severity: Informational

Context: CowEvcClosePositionWrapper.sol#L237-L245

Description: The function `_evcInternalSettle` from contract `CowEvcClosePositionWrapper` emits `CowEvcPositionClosed` with the `collateralAmount` field set to `params.collateralAmount`.

In the same execution, the wrapper reads the remaining `params.collateralVault` balance in the Inbox after `_next` returns and transfers that remaining balance back to `params.account`. This means the settlement does not necessarily consume the full `params.collateralAmount`.

As a result, the emitted `collateralAmount` can overstate the actual amount of collateral spent in the settlement.

Offchain consumers that interpret `CowEvcPositionClosed.collateralAmount` as the actual collateral spent can compute incorrect analytics or accounting. This does not affect the onchain execution path.

Recommendation: Consider to align the event field naming and documentation with what is emitted. Also could consider to emit the returned collateral amount alongside `params.collateralAmount`, so offchain consumers can derive the net collateral spent when needed.

Cow Protocol: Fixed in commit 50184a90.

Cantina Managed: Fix verified.

3.5.10 `_evcInternalSettle` swap output check can pass without any settlement output

Severity: Informational

Context: CowEvcClosePositionWrapper.sol#L205-L212

Description: The function `_evcInternalSettle` from contract `CowEvcClosePositionWrapper` checks whether the CoW settlement produced swap output by reading `borrowAsset.balanceOf(address(inbox))` after `_next` returns and reverting only when the resulting balance is zero.

This check is based on the absolute post-settlement balance of the Inbox, not on the balance increase caused by the settlement. If the Inbox already holds a non-zero balance of the borrow asset before the settlement executes, the condition `swapResultBalance == 0` is false even when the settlement sends the buy token to a different receiver or does not send any buy token to the Inbox.

The wrapper can continue the close flow without verifying that the settlement delivered any swap output to the Inbox. This makes the receiver correctness protection incomplete.

Recommendation: Consider to validate swap output using the Inbox balance delta by recording the borrow asset balance before calling `_next` and requiring it to increase after settlement. Also could consider to base the repay amount on this balance increase, so the repayment reflects swap output rather than any pre-existing Inbox balance.

Cow Protocol: Fixed in commit 75916ad3.

Cantina Managed: Fix verified.

3.5.11 `Inbox` hardcodes settlement domain separator

Severity: Informational

Context: Inbox.sol#L59-L63

Description: The constructor from contract `Inbox` computes `SETTLEMENT_DOMAIN_SEPARATOR` locally using fixed values for the EIP-712 domain name and version and the provided `settlement` address.

This does not read `ICowSettlement(settlement).domainSeparator()`. If the actual settlement contract uses different domain parameters, the computed `SETTLEMENT_DOMAIN_SEPARATOR` diverges from the settlement domain separator and `Inbox.isValidSignature` reverts with `OrderHashMismatch` for otherwise valid orders.

DRAFT

Orders that rely on the Inbox EIP-1271 signature path are not executable when the settlement domain separator differs from the hardcoded inputs.

Recommendation: Consider to read the domain separator from the settlement contract during construction and store it, so the signature verification is aligned with the deployed settlement contract.

Cow Protocol: Fixed in commit b87b0a90.

Cantina Managed: Fix verified.

3.5.12 Missing Euler vault validation in open/close wrappers

Severity: Informational

Context: CowEvcClosePositionWrapper.sol#L101-L105

Description: `OpenPositionParams.borrowVault / collateralVault` and `ClosePositionParams.borrowVault / collateralVault` are user-supplied addresses (covered by user authorization), and the wrapper logic later treats them as Euler vaults.

However, there is no onchain validation that the provided vault addresses:

- Were deployed by a trusted Euler factory / registry, or.
- Are vaults connected to the expected EVC instance.

This contradicts the project's own suggested security checklist item: *"Validate that the user cannot specify untrusted contracts (ex. arbitrary Euler vault contracts) to steal other user's funds or harm the solver"*.

In practice, because the vault addresses are user-authorized, this is primarily a **griefing / reliability** concern (e.g., forcing solvers to spend gas on calls to contracts that revert or behave unexpectedly, and only failing later in the flow). It also increases the chance that integrators accidentally route to non-Euler or wrong-EVC vaults, resulting in failed operations and harder-to-debug incidents.

Recommendation: Consider to enforce vault provenance either:

- On-chain, for example by:
  - Checking the vault is a proxy deployed by a trusted Euler factory.
  - Checking the vault reports the expected EVC address.

```
GenericFactory(factory).isProxy(vault) == true
IEVault(vault).EVC() == address(EVC)
```

- Off-chain, by allowlisting verified Euler vault addresses and rejecting untrusted vaults before producing signatures.

If this is an assumed risk, consider to explicitly document that assumption and adjust the checklist wording to avoid implying that this validation is currently enforced.

Cow Protocol: The harms of trying to restrict this at the contract level far outweigh the benefits. So no change will be made to the contracts. There is some risk to solvers and etc. but as Darek said, the risk extends beyond just the Vaults themselves (ex. if the vault is configured with a hook). So there is not too much we can do. Will update the doc checklist to ensure proper expectations are set for this in the future.

The Euler frontend does use the "Perspectives" to help protect the user and ensure they are buying into trusted vaults, so we do have that protection.

Cantina Managed: Acknowledged.

3.5.13 Unconditional max approval to the vault relayer

Severity: Informational

Context: CowEvcClosePositionWrapper.sol#L199

Description: The close wrapper always executes:

```
inbox.callApprove(params.collateralVault, VAULT_RELAYER, type(uint256).max);
```

DRAFT

This matches the common CoW Protocol trust model where orders typically rely on large allowances to the settlement/relayer. However, performing an unconditional max approval on every close has a main drawback:

- Allowance persistence / blast radius: the `Inbox` keeps a long-lived unlimited allowance for `params.collateralVault` toward `VAULT_RELAYER`. Even if this is an accepted trust assumption, it increases the impact of an unexpected allowance reuse across operations, and it is not scoped to the single close.

This is primarily a design/operational consideration rather than a correctness bug.

Recommendation: Consider to choose one of the following consistent approaches:

- Scope to operation (least privilege): approve only the required amount for the close flow and reset allowance after completion (if feasible with the call pattern and token behavior).
- Keep as-is but document: if the intention is to prevent user revocations from causing close failures, consider to explicitly document that the wrapper re-approves on each close to maintain liveness.

Cow Protocol: Fixed in commit 8186a597.

Cantina Managed: Fix verified.

3.5.14 `toAmount` parameter in `CowEvcCollateralSwapWrapper` is unused

Severity: Informational

Context: CowEvcCollateralSwapWrapper.sol#L86-L87

Description: `toAmount` is documented as: *"The amount of toVault traded out. Same as `buyAmount` in the CoW order"*, but the wrapper does not use this value.

As a result, the wrapper itself does not enforce any min-output / slippage condition based on `toAmount`, and correctness relies entirely on the CoW Settlement path to ensure that the executed swap respects user intent.

If a solver (or any authorized caller, depending on the deployment/auth model) can execute a wrapper call that results in:

- Settlement not applying the expected limit, or.
- The swap output being routed incorrectly, the wrapper would not detect it via `toAmount`, despite the parameter suggesting such protection exists.

Even if, in practice, settlement always applies slippage constraints for valid orders, leaving an unused buyAmount like parameter increases integrator confusion and makes it easier to accidentally assume an on-chain min-out enforcement that is not present.

Recommendation: Consider to either:

- Enforce `toAmount` in the wrapper (e.g., require the `Inbox` / relevant receiver balance delta of the destination token is `>= toAmount` after settlement), or.
- Remove / rename / document `toAmount` to avoid implying it is validated on-chain, and explicitly document that slippage/min-output is enforced exclusively by settlement/order constraints.

Cow Protocol: Fixed in commit d682fd45.

Cantina Managed: Fix verified.

3.5.15 No interfaces provided for integrators

Severity: Informational

Context: CowWrapper.sol#L78

Description: `ICowWrapper` is declared inside `CowWrapper.sol` alongside implementation logic. This matches the stated goal of keeping the wrapper easy to copy/paste as a standalone unit.

DRAFT

However, co-locating interfaces with implementation can reduce discoverability and standard "entry point" expectations for integrators and auditors, who often look for interface definitions under a dedicated `interfaces/` directory.

Recommendation: Consider to keep the current "single-file copy/paste" design, but also provide a mirrored `interfaces/ICowWrapper.sol` (or similar) as a canonical reference location for interface definitions and NatSpec, if repo ergonomics for integrators/auditors is a priority.

Cow Protocol: Acknowledged.

Cantina Managed: Acknowledged.

DRAFT